NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA

Department of Computer Science and Engineering

Project Title: AI-Based Smart Quiz System.

Course Code: CSE 4204
Course Title: Mobile Computing Lab

Submitted By:

Team Name : CSE4204-8A-T03
Section : 8A

GitHub Repository: https://github.com/sudipto224/AI-based-Smart-Quiz-System

Submitted to:

Md. Riaz Mahmud
Designation: Assistant Professor
Department of CSE

Date of Submission: 15/06/2026

TEAM INFORMATION

Official Team Name	CSE4204-8A-T03
Section	8A
Project Title	AI-Based Smart Quiz System
GitHub Repository	https://github.com/sudipto224/AI-based-Smart-Quiz-System

Team Information:

Role	Full Name	Student ID
Team Leader + Backend	Sudipto Das	11220320830
Frontend	Chaon Das	11210320629
Artificial Intelligence	Rakibur Rahman Mishan	11220320829
Database	Aditi Roy	11220320846

Project Information

Project Overview : The AI-Based Smart Quiz System is a web-based online assessment platform built with Laravel 12, MySQL, and vanilla JavaScript. It is designed for academic institutions to conduct efficient, secure, and engaging quizzes. The system supports two user roles: Teacher and Student.

Teachers can create courses, manually add multiple-choice questions, and set a configurable timer per question. For a designated course, teachers can use an AI feature that generates complete MCQs automatically. By simply entering a topic, the system calls the Google Gemini API and returns a question, four options, the correct answer, and an explanation. The form is then auto-filled for the teacher to review and save.

Students can take timed quizzes for any available course. A countdown timer is shown for each question, and the quiz auto-submits if time runs out. After submission, students instantly see their score, the correct answers, and explanations for the questions they got wrong.

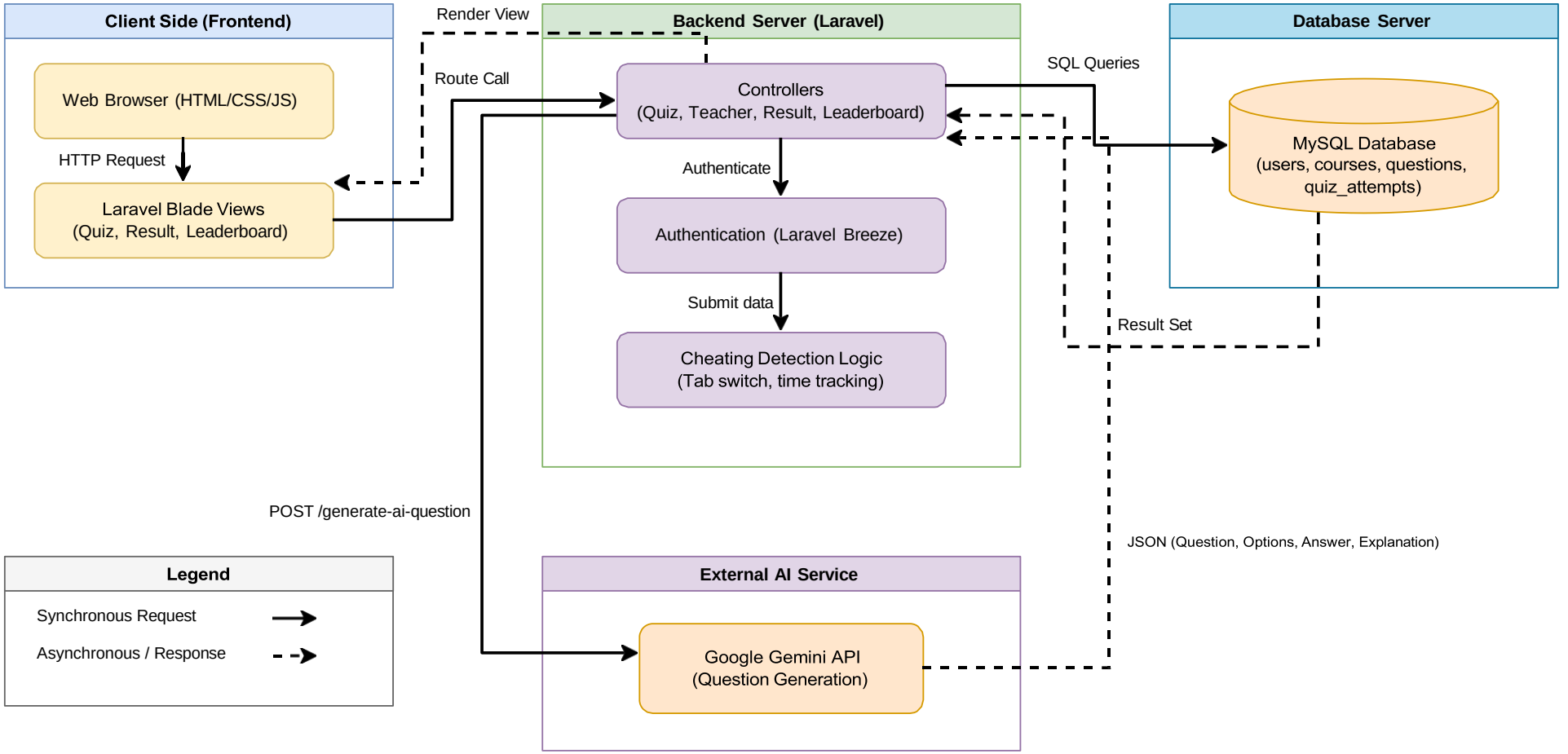
A unique AI-powered cheating detection system is active during quizzes for the designated course. Using JavaScript's visibilitychange event, the system counts how many times a student switches away from the quiz tab. It also calculates the average time spent per question. If the student switches tabs at least once or averages less than five seconds per question, the attempt is marked as suspicious. Teachers can review all flagged attempts in a dedicated panel.

The system also includes a leaderboard that ranks students by highest score and shortest completion time. Suspicious attempts are excluded from the leaderboard to ensure fairness.

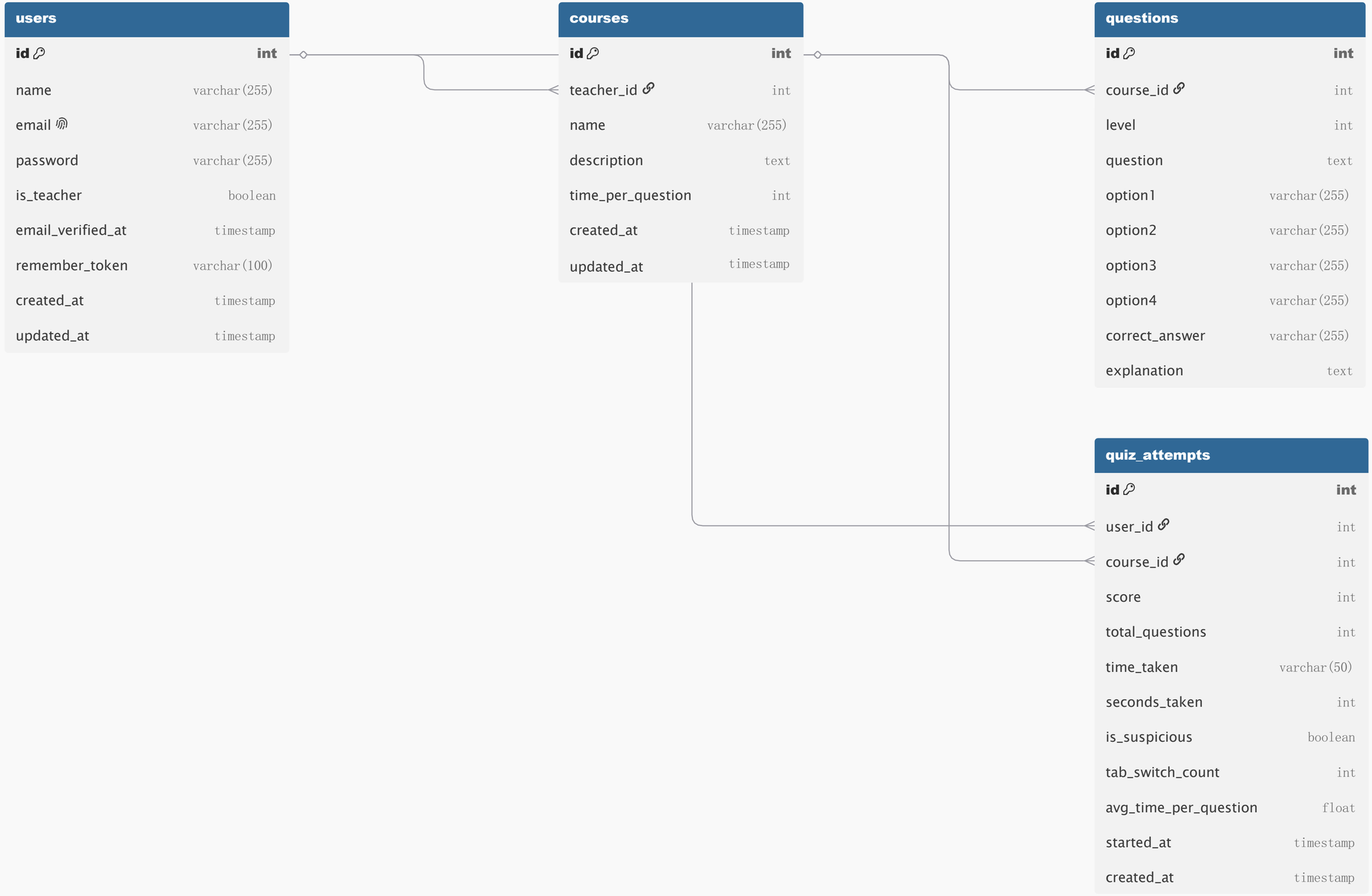
Technology used includes HTML5, custom CSS, and vanilla JavaScript for the frontend; Laravel 12 with PHP 8.2 for the backend; MySQL 8.0 for the database; and Google Gemini API for AI question generation. Diagrams are created with draw.io and dbdiagram.io and the project is managed with Git and GitHub.

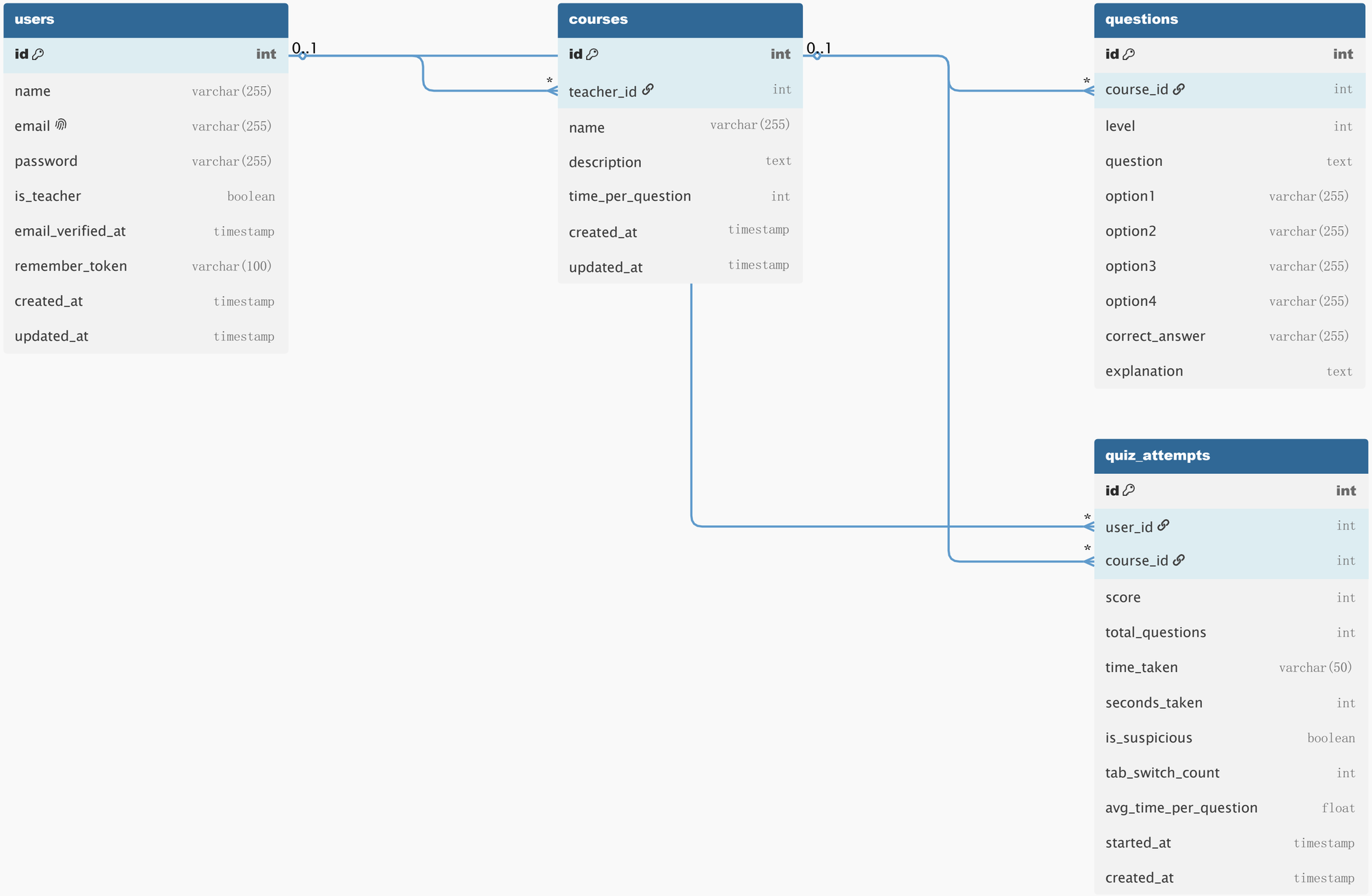
This project is developed as part of the CSE4204 course at Northern University of Business and Technology, Khulna. It covers the complete quiz lifecycle, two AI features, and a responsive user interface. Future enhancements such as real-time face proctoring, progressive web app support, and teacher analytics dashboards are not included in the current scope.

1. System Architecture Diagram :



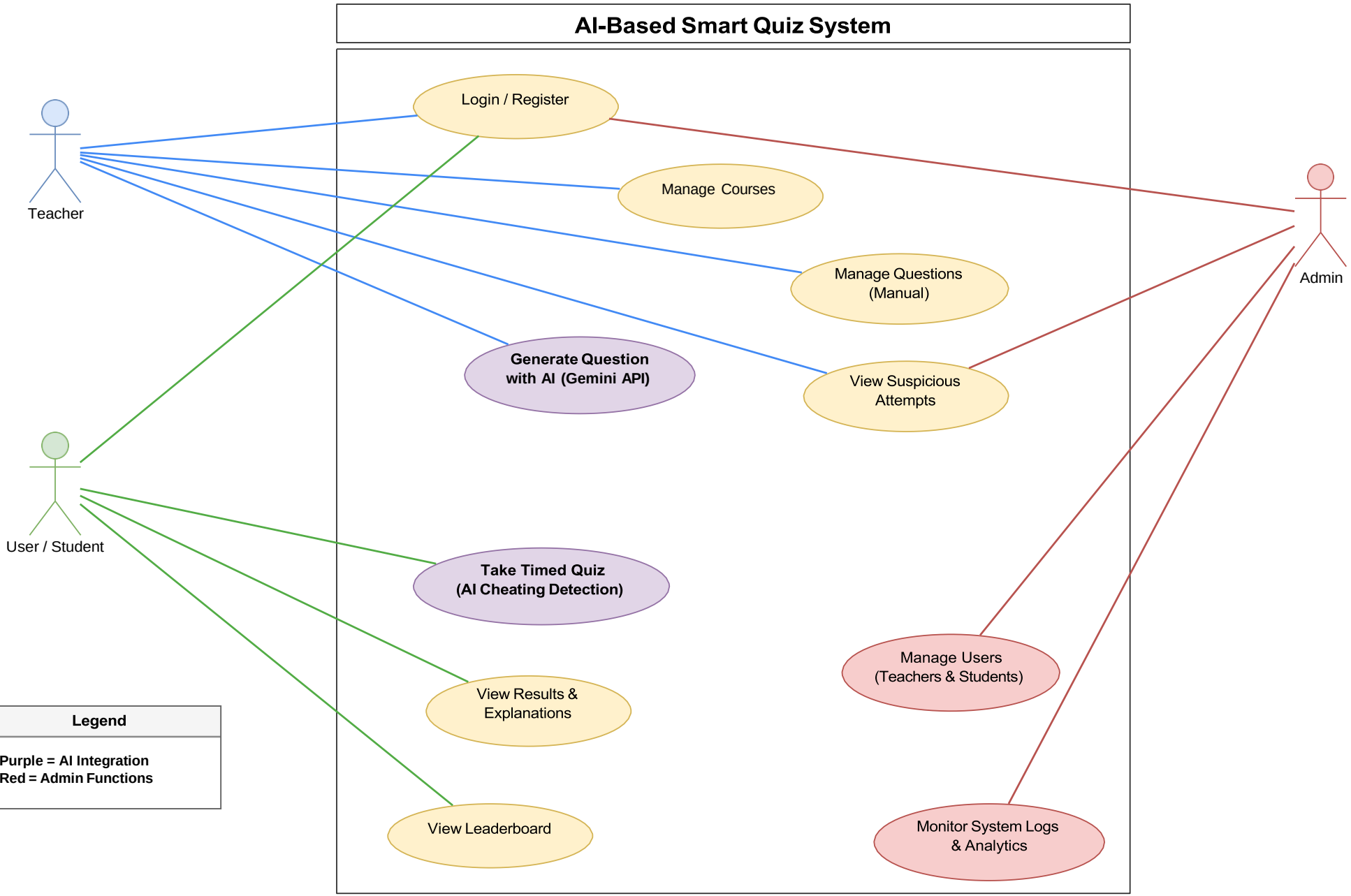
2. ER Diagram (Database Design) :



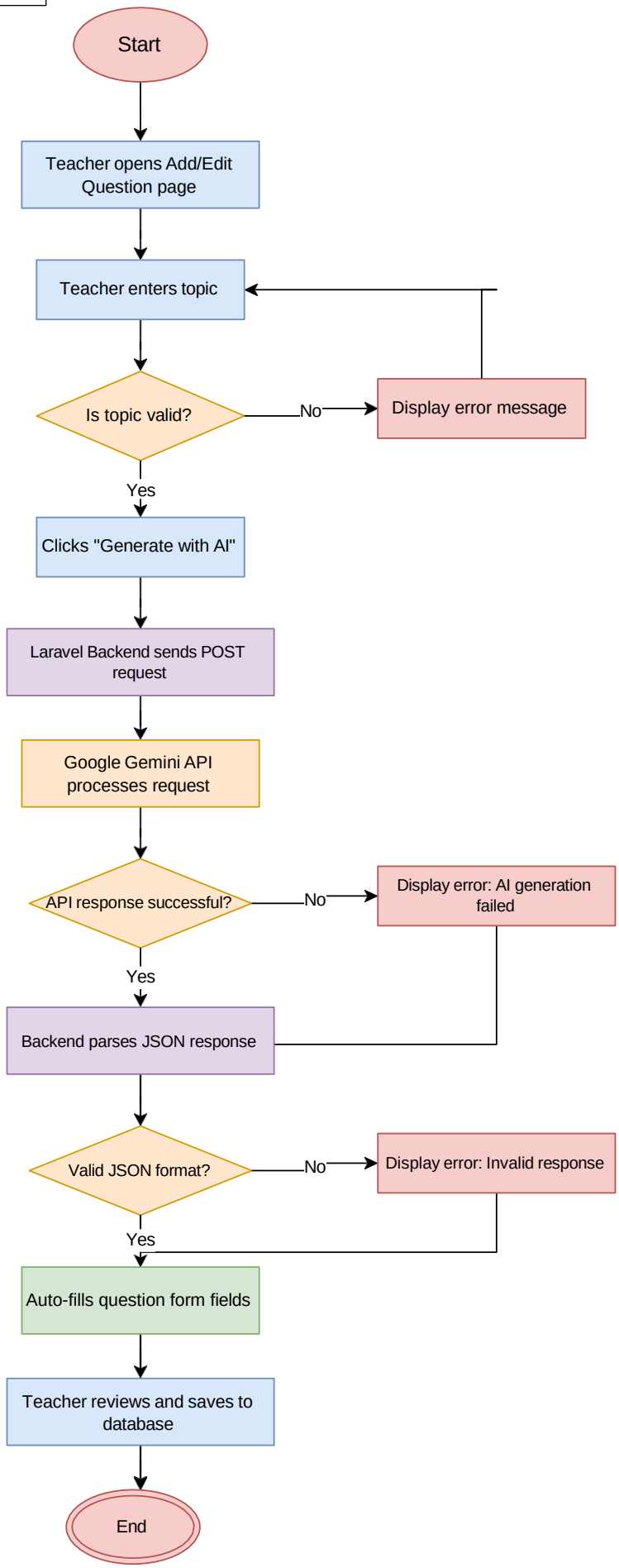

users
id int
name varchar(255)
email varchar(255)
password varchar(255)
is_teacher boolean
email_verified_at timestamp
remember_token varchar(100)
created_at timestamp
updated_at timestamp
courses
id int
teacher_id int
name varchar(255)
description text
time_per_question int
created_at timestamp
updated_at timestamp
questions
id int
course_id int
level int
question text
option1 varchar(255)
option2 varchar(255)
option3 varchar(255)
option4 varchar(255)
correct_answer varchar(255)
explanation text
quiz_attempts
id int
user_id int
course_id int
score int
total_questions int
time_taken varchar(50)
seconds_taken int
is_suspicious boolean
tab_switch_count int
avg_time_per_question float
started_at timestamp
created_at timestamp
0..1
*
0..1
*
*
*
dbdiagram.io

3. Updated Use Case Diagram :

System Use Case Diagram - AI-Based Smart Quiz System



4. Activity Diagram:



5. API Design Document – AI-Based Smart Quiz System

5.1 Introduction

This document describes the API design for the AI-Based Smart Quiz System, a Laravel-based web application supporting two user roles: Teacher (is_teacher = 1) and Student (is_teacher = 0). The system uses Laravel‟s session-based authentication rather than token-based mechanisms. All routes are standard Laravel web routes protected by the „auth‟ and role-checking middleware. There are no Bearer tokens or API keys passed in request headers from the client side; instead, the authenticated session cookie is used to identify and authorize users. The APIs are grouped into the following categories: Authentication, Teacher – Course Management, Teacher – Question Management, AI Question Generation, Student Quiz, Leaderboard, Suspicious Attempt Monitoring, and User Profile.

5.2 Authentication APIs

Method	Endpoint	Purpose	Input Data	Expected Output
POST	/register	Register a new user (Teacher or Student)	name, email, password, password_confirmation, is_teacher (0 or 1)	Redirects to dashboard on success; returns validation errors on failure
POST	/login	Authenticate an existing user and start a session	email, password, remember (optional boolean)	Redirects to role-based dashboard on success; returns error message on failure
POST	/logout	Terminate the authenticated user session	None (uses active session token via CSRF)	Redirects to login page; session is invalidated

5.3 Teacher – Course Management APIs

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/teacher/courses	Retrieve and display all courses belonging to the authenticated teacher	None	HTML view listing all courses with Edit and Delete options
POST	/teacher/courses	Create a new course	title (string), description (text), timer_minutes (integer – configurable quiz timer)	Redirects to course list on success; validation errors returned on failure
GET	/teacher/courses/{id}/edit	Show the edit form pre-populated with the selected course‟s data	URL parameter: id (course ID)	HTML edit form with existing course data populated
PUT	/teacher/courses/{id}	Update an existing course‟s details	URL parameter: id; Body: title, description, timer_minutes	Redirects to course list on success; validation errors on failure

Method	Endpoint	Purpose	Input Data	Expected Output
DELETE	/teacher/courses/{id}	Permanently delete a course and its associated questions	URL parameter: id (course ID)	Redirects to course list with a success message

5.4 Teacher – Question Management (Manual) APIs

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/teacher/courses/{course_id}/questions	List all MCQ questions associated with a specific course	URL parameter: course_id	HTML view listing all questions with Edit and Delete options

POST	/teacher/courses/{course_id}/questions	Add a new MCQ question to a specific course	URL parameter: course_id; Body: question_text, option_a, option_b, option_c, option_d, correct_answer	Redirects to question list on success; validation errors on failure
Method	**Endpoint**	**Purpose**	**Input Data**	**Expected Output**
			(a/b/c/d), explanation	
GET	/teacher/questions/{id}/edit	Display the edit form for a specific question	URL parameter: id (question ID)	HTML edit form pre-filled with the question‟s data
PUT	/teacher/questions/{id}	Update the content of an existing MCQ question	URL parameter: id; Body: question_text, option_a, option_b, option_c, option_d, correct_answer	Redirects to question list on success; validation errors on failure

			, explanation	
DELETE	/teacher/questions/{id}	Delete a specific MCQ question permanently	URL parameter: id (question ID)	Redirects to question list with a success confirmation message

5.5 AI Question Generation API

The teacher enters a topic, and the server calls the Google Gemini API to generate a complete MCQ. The result is returned as JSON so the teacher can review and save it.

Method	Endpoint	Purpose	Input Data	Expected Output
POST	/teacher/generate-ai-question	Call the Google Gemini API with a given topic and return a generated MCQ in JSON format	Body: topic (string – the subject for which the question should be generated), course_id (to verify it is Mobile Computing Lab)	JSON response: { "question": "...", "options": ["A. ...", "B. ...", "C. ...", "D. "], "correct_answer": "A", "explanation": " " }

5.6 Student Quiz APIs

These routes allow students to load and take timed quizzes. For the “Mobile Computing Lab” course, the quiz page embeds hidden fields to capture cheating detection data. The timer is configurable per course (set by the teacher). Upon submission, the server evaluates answers, computes average time per question, and flags the attempt as suspicious if warranted.

Method	Endpoint	Purpose	Input Data	Expected Output

GET	/quiz/{course_id}	Load the quiz page for a specific course, including questions, timer	URL parameter: course_id	HTML quiz page with: all MCQ questions, a countdown timer (from timer_minutes), hidden fields: tab_switch_count
Method	**Endpoint**	**Purpose**	**Input Data**	**Expected Output**
		configuration, and hidden fields for cheating detection		(default 0, incremented by JavaScript visibilitychange event) and quiz_start_time (Unix timestamp in milliseconds)

POST	/quiz/submit	Submit quiz answers. Server calculates score, average time per question, and determines whether the attempt is suspicious	Body: course_id, answers[] (array of selected options per question), tab_switch_count (integer), quiz_start_time (integer, milliseconds). Cheating rule: attempt is flagged as suspicious if tab_switch_count > 0 OR avg_time_per_question < 5 seconds	Saves QuizAttempt record with score, time_taken, tab_switch_count, avg_time_per_question, is_suspicious flag; Redirects to /result/{attempt_id}
GET	/result/{attempt_id}	Display the result page for a completed	URL parameter: attempt_id	HTML result page showing: total score, per-question breakdown (student‟s
Method	**Endpoint**	**Purpose**	**Input Data**	**Expected Output**
		quiz attempt		answer vs. correct answer), explanations for each question, and total time taken

5.7 Leaderboard API

The leaderboard ranks students by score (descending) and time taken (ascending) for a given course. Attempts flagged as suspicious (is_suspicious = 1) are excluded from the ranking to ensure integrity.

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/leaderboard/{course_id}	Display the ranked leaderboard for a specific course, excluding suspicious attempts	URL parameter: course_id	HTML leaderboard view showing: Rank, Student Name, Score, Time Taken; ordered by score DESC and time_taken ASC; all records where is_suspicious = 0

5.8 Teacher – Suspicious Attempts Monitoring API

This endpoint allows teachers to review all quiz attempts flagged as suspicious for the “Mobile Computing Lab” course. An attempt is marked suspicious when tab_switch_count > 0 OR avg_time_per_question < 5 seconds.

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/teacher/suspicious-attempts	List all flagged quiz attempts for the "Mobile Computing Lab" course for teacher review	None (teacher-authenticated session required)	HTML view listing all suspicious attempts with: Student Name, Score, tab_switch_count, avg_time_per_question (in seconds), attempted_at (datetime)

5.9 User Profile APIs

Method	Endpoint	Purpose	Input Data	Expected Output
GET	/profile	Display the authenticated user‟s profile information	None (uses active session)	HTML profile page showing: name, email, role (Teacher or Student)
PUT	/profile	Update the authenticated user‟s profile details	Body: name (string), email (string), password (string, optional), password_confirmation (required if password provided)	Redirects to profile page with a success message; validation errors returned on failure

Note on Cheating Detection Data Transmission

Cheating detection data for the "Mobile Computing Lab" course is transmitted via hidden HTML form fields embedded in the quiz page. When the quiz page loads (GET /quiz/{course_id}), two

hidden input fields are rendered in the form: (1) quiz_start_time – populated by JavaScript with Date.now() at the moment the student begins the quiz, recorded in milliseconds; and (2) tab_switch_count – initialized to 0 and incremented in real time by a JavaScript event listener on the document.visibilitychange event each time the student switches away from the quiz tab. When the student submits the quiz (POST /quiz/submit), both values are posted along with the answers array and course_id as standard HTML form fields. The server then computes avg_time_per_question = (submission_time – quiz_start_time) / 1000 / total_questions, and applies the cheating detection rule: if tab_switch_count > 0 OR avg_time_per_question < 5, the attempt is saved with is_suspicious = 1 and excluded from the leaderboard.

6. AI Integration Workflow

6.1 Why AI is Needed

The AI-Based Smart Quiz System incorporates artificial intelligence to address two fundamental challenges faced in traditional quiz management and examination environments.

First, manual question creation is a time-consuming and resource-intensive process. Teachers are required to individually compose each question, formulate four plausible answer options, identify the correct response, and write an explanatory note — all for every topic they wish to assess. In a course with a broad syllabus, this places a significant burden on the instructor and limits the volume and variety of questions that can be realistically produced. Automating this process through AI enables teachers to generate complete, well-structured MCQs in seconds by simply providing a topic keyword.

Second, detecting academic dishonesty during online quizzes is difficult to accomplish manually. When students take quizzes in an unproctored environment, behaviours such as rapidly switching browser tabs to consult external resources or completing questions at an unnaturally fast pace are difficult to identify without automated assistance. The system addresses this by integrating client-side behavioural monitoring to detect and flag suspicious quiz attempts, thereby preserving the integrity of the assessment process.

6.2 AI Service and Model Used

AI Question Generation: The system integrates with the Google Gemini API, using the model gemini-2.0-flash. This model is selected for its speed, efficiency, and ability to produce well-formed, contextually accurate multiple-choice questions from a brief topic prompt. Communication with the Gemini API is handled server-side within the Laravel application to ensure API key security and consistent response parsing.

Cheating Detection: The cheating detection mechanism does not rely on an external AI service. It is implemented using custom JavaScript logic embedded in the quiz page, combined with server-side validation in Laravel. The JavaScript visibilitychange event tracks tab-switching behaviour, while the server calculates average time per question from the submitted quiz

timestamps. This rule-based logic effectively identifies behavioural anomalies without requiring a third-party AI provider.

6.3 Input to the AI System

For the AI Question Generation feature, the teacher provides a single input: a topic string. This is a short text phrase describing the subject area for which the MCQ should be generated (for example, “4G LTE Architecture” or “Bluetooth Protocol”). The teacher enters this topic into a dedicated input field on the question management page for the Mobile Computing Lab course. The Laravel controller then constructs a structured prompt around this topic and submits it to the Google Gemini API.

6.4 Output from the AI System

The Google Gemini API returns a response that the Laravel backend parses into a structured JSON object. The expected output format is as follows:

{

"question": "What is the primary function of the MAC layer in a mobile network?",

"options": [

"A. To manage physical radio transmission",

"B. To control medium access and scheduling",

"C. To assign IP addresses to mobile devices",

"D. To encrypt application-layer data"

],

"correct_answer": "B",

"explanation": "The MAC (Medium Access Control) layer is responsible for controlling how devices access the shared radio medium, including scheduling and resource allocation."

}

This JSON response is returned to the front-end via the Laravel controller, and the values are automatically populated into the corresponding form fields — question text, four option inputs, the correct answer selector, and the explanation field — allowing the teacher to review, adjust if necessary, and save the question.

6.5 How AI Improves the Project

The integration of AI into the Smart Quiz System delivers the following benefits:

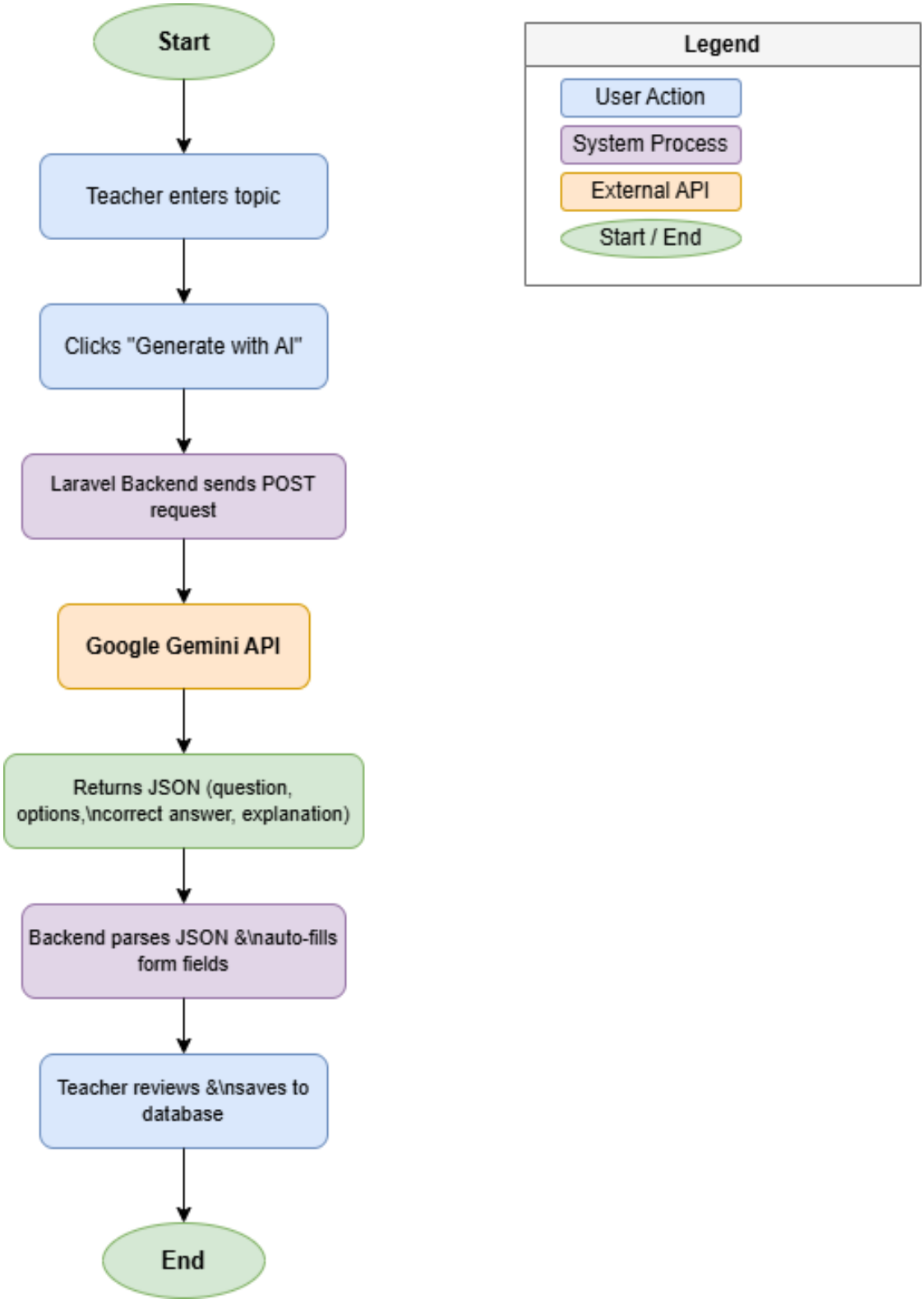
- Efficiency: Teachers can generate a complete, ready-to-use MCQ within seconds by entering a single topic keyword, eliminating the need to manually compose questions, options, and explanations from scratch. This significantly reduces question preparation time.
- Quality and Consistency: The Gemini model produces grammatically correct, contextually relevant questions with clearly differentiated answer options and accurate explanations, maintaining a consistent standard of question quality across all AI-generated content.
- Scalability: The system can support a large and growing question bank without placing additional workload on teachers. As course content expands, AI generation allows rapid creation of questions covering new topics.
- Academic Integrity: The automated cheating detection mechanism flags attempts involving tab switching or abnormally fast responses, allowing the system to exclude suspicious submissions from the leaderboard and providing teachers with an audit trail of flagged attempts for review.
- Improved Student Experience: Students benefit from a richer and more varied question pool, receive immediate explanations after each quiz, and participate in a fair and integrity-protected assessment environment.

6.6 AI Question Generation Workflow

The following steps describe the end-to-end workflow for AI-assisted question generation in the system:

1. The teacher logs into the system with their credentials and navigates to the question management page for the Mobile Computing Lab course.
2. The teacher clicks the “Generate Question with AI” button, which reveals a topic input field on the page.
3. The teacher types a topic string (e.g., “OFDM in 4G Networks”) into the input field and submits the request.
4. The browser sends a POST request to the Laravel route /teacher/generate-ai-question, passing the topic string and the course identifier in the request body.
5. The Laravel controller receives the request, constructs a structured natural-language prompt incorporating the topic, and sends it to the Google Gemini API (model: gemini-2.0-flash) using an authenticated HTTP request with the server-stored API key.
6. The Gemini API processes the prompt and returns a JSON-formatted response containing the question text, an array of four answer options, the correct answer identifier, and an explanation.
7. The Laravel controller parses the API response and returns the structured JSON data to the browser.
8. JavaScript on the question form page receives the JSON response and automatically populates the form fields: question text, option A through D, correct answer, and explanation.
9. The teacher reviews the auto-filled question, makes any necessary edits, and clicks “Save” to persist the question to the database via the standard POST /teacher/courses/{course_id}/questions route.
10. The question is saved and becomes immediately available for inclusion in student quizzes for the Mobile Computing Lab course.

6.7 Workflow Diagram :



References :

1.Laravel Documentation. (2026). Laravel 12.x – The PHP Framework for Web Artisans. Retrieved from https://laravel.com/docs/12.x

2.MySQL Documentation. (2026). MySQL 8.0 Reference Manual. Oracle Corporation. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/

3.Google Gemini API Documentation. (2026). Gemini API – Google AI for Developers. Retrieved from https://ai.google.dev/gemini-api/docs

4.Diagrams.net (Draw.io) Documentation. (2026). User Guide – Diagrams.net. Retrieved from https://www.diagrams.net/doc

5.dbdiagram.io Documentation – DBML Reference. Holistics.io. Available at: https://docs.dbdiagram.io